MCT 332/MCT334 Design of MCT 2
Spring 2026
Raspberry PI Setup, Pub and Sub and GPIOs

Lab Objectives:

I. Raspberry Pi and ROS2 Setup
   - Install Ubuntu 24.04 on Raspberry Pi 5.
   - Install and configure ROS 2 Jazzy environment.

II. Remote Connection Setup
   - Enable and configure SSH on Raspberry Pi.
   - Establish a remote connection from the laptop to Raspberry Pi through the same network.

III. ROS2 Publisher and Subscriber Implementation
   - Create a ROS 2 workspace and package.
   - Develop a Python publisher node to publish messages to a topic.
   - Develop a Python subscriber node to receive and display messages from the topic.

IV. GPIO Configuration and Control
   - Install and configure GPIO libraries on Raspberry Pi.
   - Write a Python script to control GPIO pins and blink an LED.

V. IMU Sensor Integration
   - Connect the MPU6050 IMU sensor using the I2C interface.
   - Detect the sensor address and verify the connection.
   - Develop a Python script to read accelerometer and gyroscope data from the IMU

Steps:

➢ *Ubuntu24.04 Installation*

First, we need to install Ubuntu24.04 and ROS Jazzy on our Raspberry PI5.

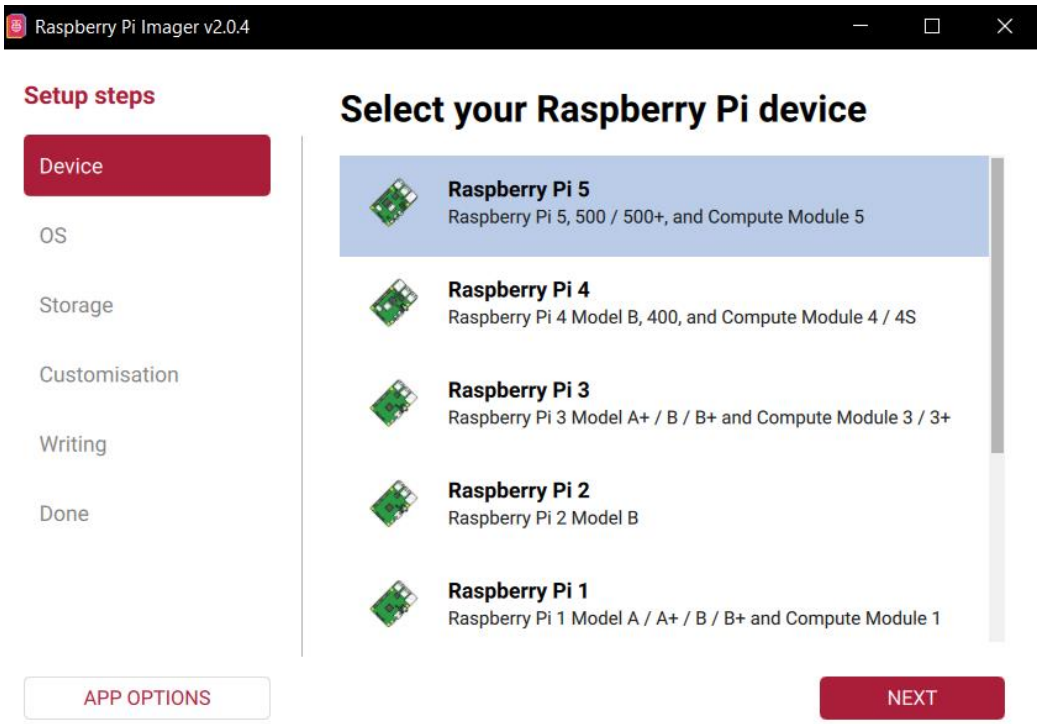
I. Download and Install ***Raspberry PI Imager*** from the following link:

   https://downloads.raspberrypi.com/imager/imager_latest.exe

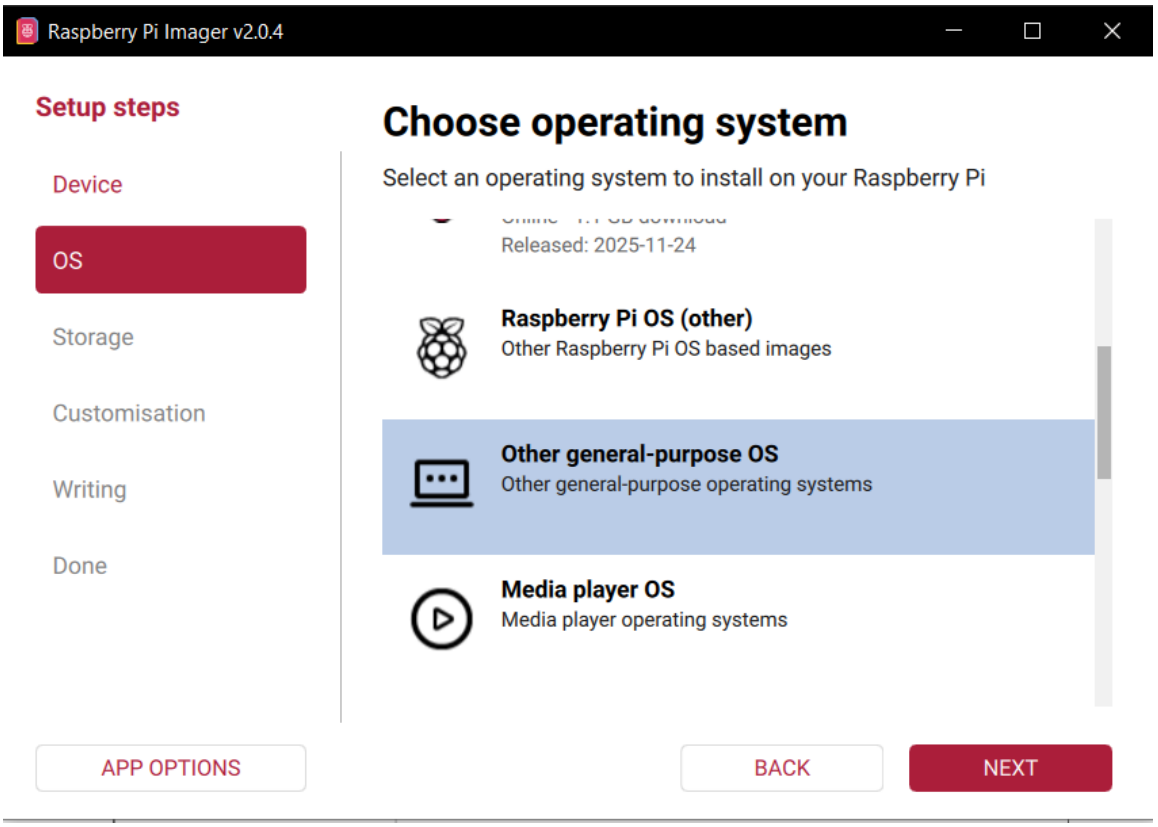
II. Choose SD card 32 GB (you can get 16 GB, but it will be bit

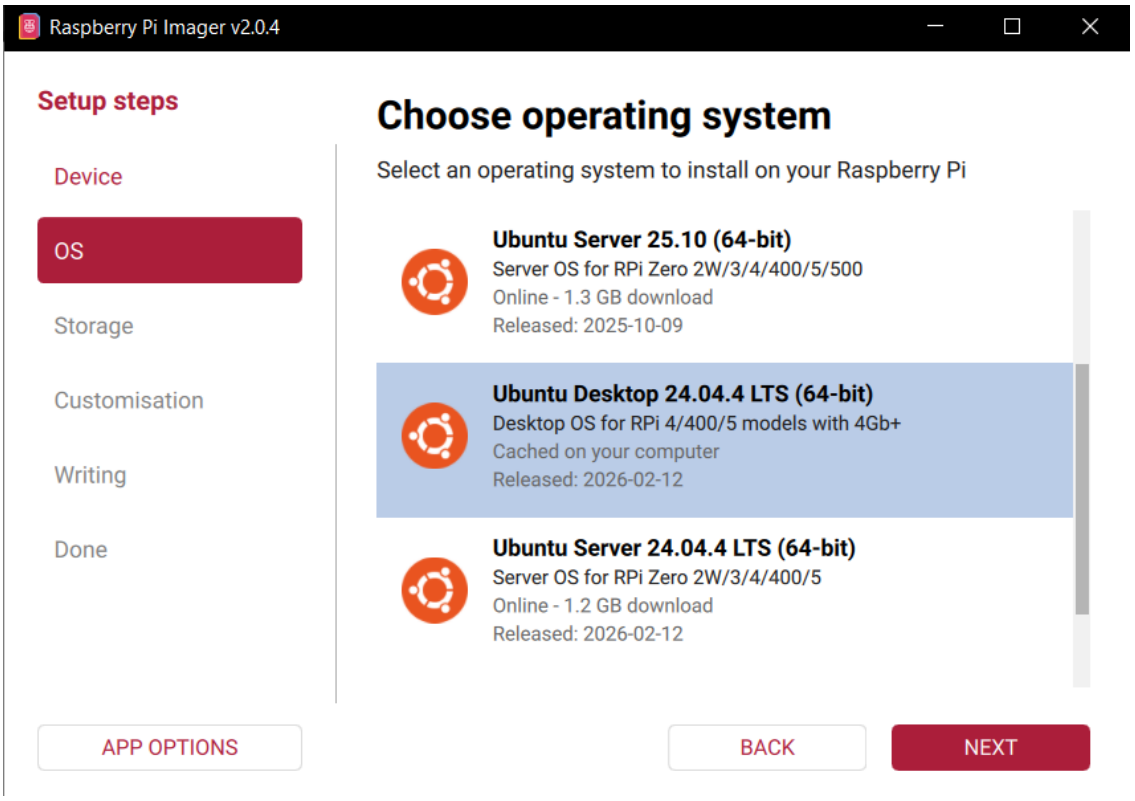
slower), insert in your laptop, then open Raspberry PI Imager.

III. Choose from ***Device*** your Raspberry model. In our case, we will choose *"Raspberry Pi 5"*

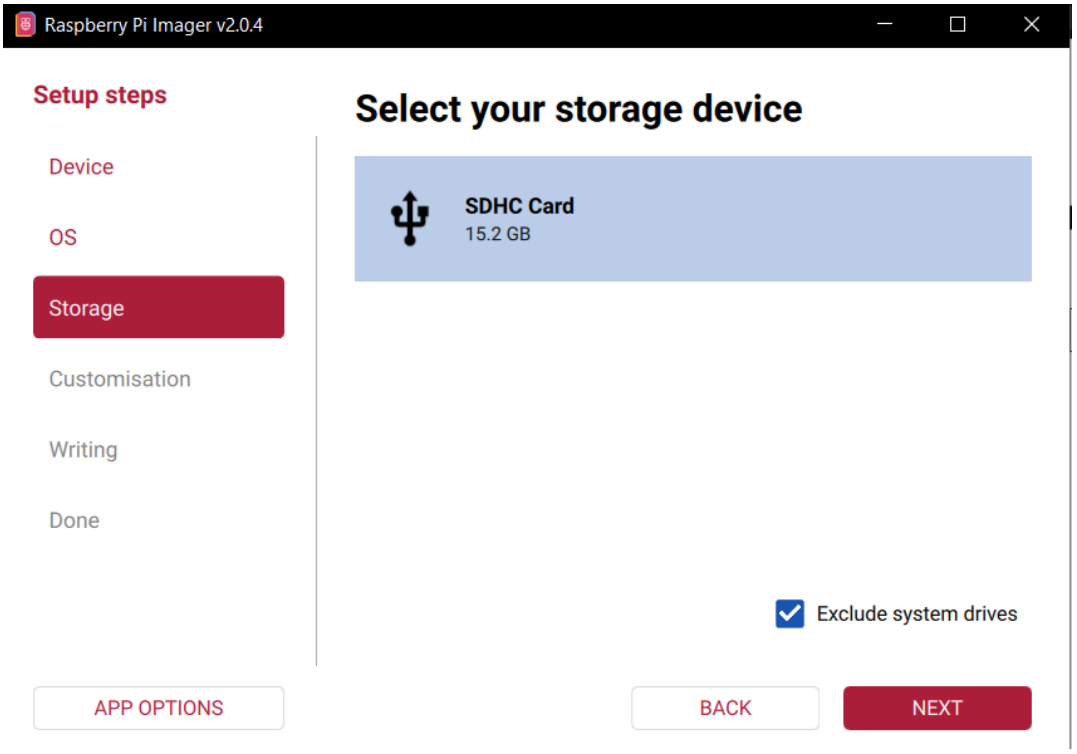


IV. From ***OS***, choose *Other general-purpose OS*, then *Ubuntu*, Choose ***"Ubuntu Desktop 24.04.4 LTS (64-bit)"***

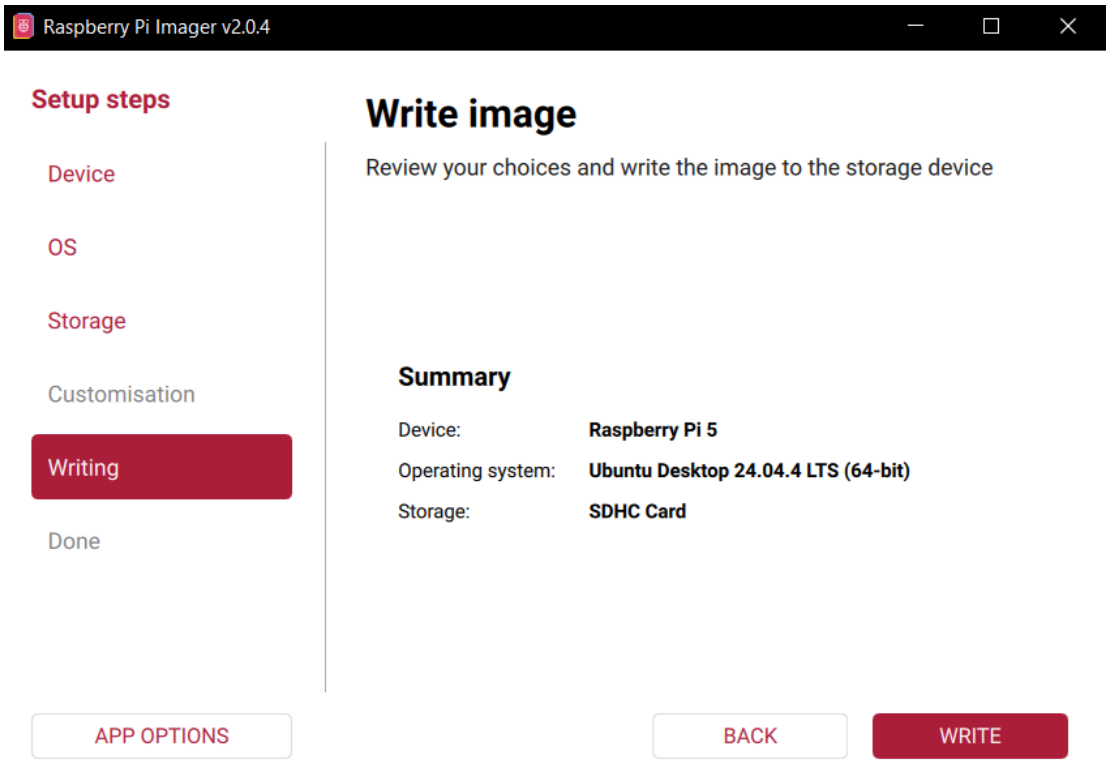


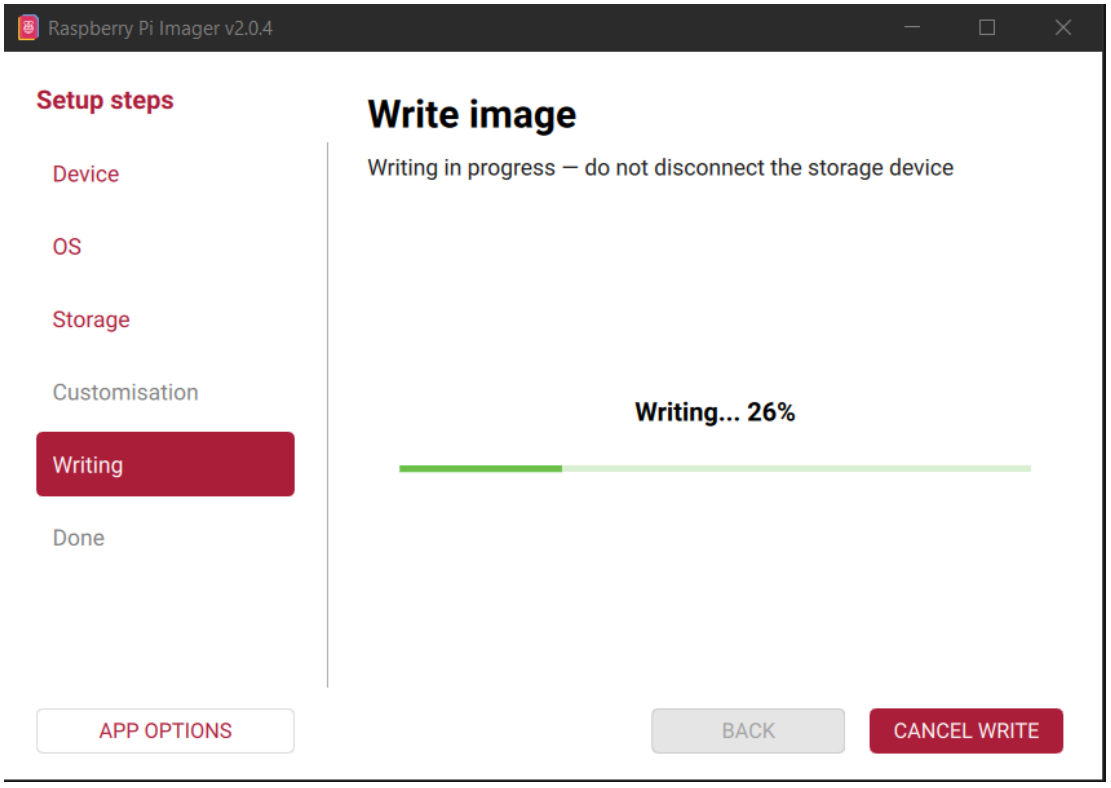


V. Choose the SD card you inserted.

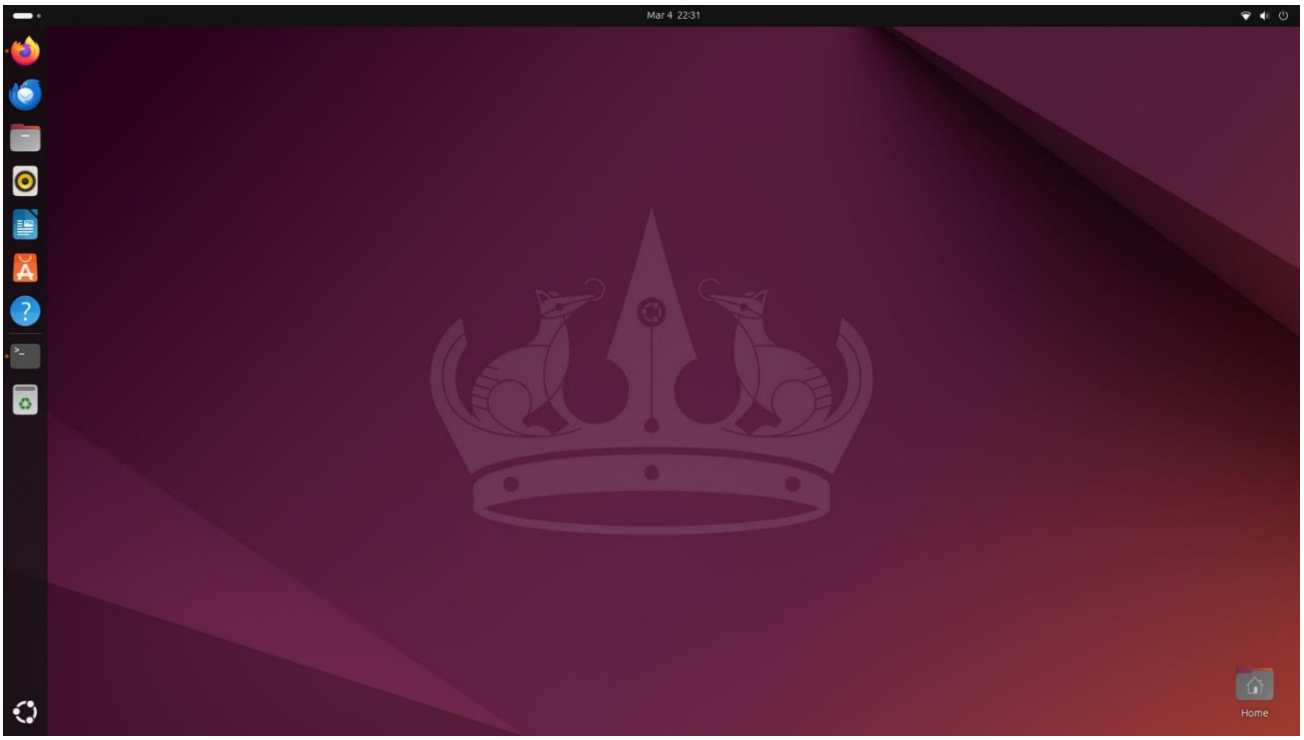


VI. Then, click on write and wait till it is done.





VII. After finishing, remove the SD card from your laptop, insert it in Raspberry Pi. Connect your Raspberry Pi on external screen and then set up Ubuntu.



➢ *SSH Setup*

In the second step, we need to connect the laptop to the Raspberry Pi through SSH instead of using external screen.

I. Open terminal window on Ubuntu (**CTRL+SHIFT+T**), then run the following commands to install SSH server.

sudo apt update

sudo apt install openssh-server

II. Enable the SSH server

sudo systemctl enable ssh

```
design2ta@design2ta-desktop:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/system
d/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/sshd.service → /usr/lib/systemd/system/ssh.s
ervice.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /usr/l
ib/systemd/system/ssh.service.
design2ta@design2ta-desktop:~$
```

III. Start SSH service

sudo systemctl start ssh

```
design2ta@design2ta-desktop:~$ sudo systemctl start ssh
```

IV. To check the ssh is enabled, run the following command:

sudo systemctl status ssh

```
design2ta@design2ta-desktop:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enab>
     Active: active (running) since Wed 2026-03-04 22:02:08 EET; 9s ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 35563 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCES>
   Main PID: 35565 (sshd)
      Tasks: 1 (limit: 9064)
     Memory: 1.2M (peak: 1.6M)
        CPU: 36ms
     CGroup: /system.slice/ssh.service
             └─35565 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Mar 04 22:02:08 design2ta-desktop systemd[1]: Starting ssh.service - OpenBSD Se>
Mar 04 22:02:08 design2ta-desktop sshd[35565]: Server listening on 0.0.0.0 port>
Mar 04 22:02:08 design2ta-desktop sshd[35565]: Server listening on :: port 22.
Mar 04 22:02:08 design2ta-desktop systemd[1]: Started ssh.service - OpenBSD Sec>
lines 1-18/18 (END)
```

To close this document, press ***"q"***.

V. Now, we need to connect laptop and Raspberry through the same WIFI. (**N.B.** *To connect the laptop to the Raspberry Pi every time without reconfiguring the IP address, ensure that both devices are connected to the same Wi-Fi network that was used during the initial setup. This allows the laptop to reach the Raspberry Pi automatically using the same network configuration.*)

VI. Get the **IP address** of your Raspberry PI, run the following command:

hostname -I

```
design2ta@design2ta-desktop:~$ hostname -I
192.168.1.73
design2ta@design2ta-desktop:~$
```

VII. Now, open command window on your laptop, run the following command to connect to your Raspberry PI

*ssh **username@IPAdd***

N.B. Change ***username*** with your username of ubuntu and ***IPAdd*** with the IP address of Raspberry PI you just got

For example, (in my case)

ssh design2ta@192.168.11.144

In the first time, it will ask if you want to connect to this IP address, type ***yes***. After that, it will ask about the password, write the password of your Ubuntu that you have set up during Ubuntu installation step. Now, you are connected

successfully.

```
design2ta@design2ta-desktop: ~
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>ssh design2ta@192.168.11.44

C:\WINDOWS\system32>ssh design2ta@192.168.11.144
The authenticity of host '192.168.11.144 (192.168.11.144)' can't be established.
ED25519 key fingerprint is SHA256:3YV3/LQGzAjjF7/nNncoxXCk3oHfGeqNjA6ipB1Nkjs.
This host key is known by the following other names/addresses:
    C:\Users\DELL/.ssh/known_hosts:4: 192.168.1.73
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.11.144' (ED25519) to the list of known hosts.
design2ta@192.168.11.144's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-1048-raspi aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Mar  5 00:46:32 2026 from 192.168.1.74
design2ta@design2ta-desktop:~$
```

➢ *ROS Jazzy Setup*

Follow the steps on the following link to install ROS Jazzy using your Terminal

https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html

You can write on your command window. For paste in the terminal, you should use keys CTRL+Shift+V (only run the parts mentioned in the images in the next step)

I. Install ROS Jazzy Desktop

Set locale

Make sure you have a locale which supports `UTF-8`. If you are in a minimal environment (such as a docker container), the locale may be something minimal like `POSIX`. We test with the following settings. However, it should be fine if you're using a different UTF-8 supported locale.

```
locale  # check for UTF-8

sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

locale  # verify settings
```

Enable required repositories

You will need to add the ROS 2 apt repository to your system.

First ensure that the Ubuntu Universe repository is enabled.

```
sudo apt install software-properties-common
sudo add-apt-repository universe
```

Now add the ROS 2 GPG key with apt.

```
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros
```

Then add the repository to your sources list.

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] ht
```

Install ROS 2

Update your apt repository caches after setting up the repositories.

```
sudo apt update
```

ROS 2 packages are built on frequently updated Ubuntu systems. It is always recommended that you ensure your system is up to date before installing new packages.

```
sudo apt upgrade
```

Desktop Install (Recommended): ROS, RViz, demos, tutorials.

```
sudo apt install ros-jazzy-desktop
```

If you are going to build ROS packages or otherwise do development, you can also install the development tools:

```
sudo apt update && sudo apt install ros-dev-tools
```

```
design2ta@design2ta-desktop: ~
The authenticity of host '192.168.11.144 (192.168.11.144)' can't be established.
ED25519 key fingerprint is SHA256:3YV3/LQGzAjjF7/nNncoxXCk3oHfGeqNjA6ipB1Nkjs.
This host key is known by the following other names/addresses:
    C:\Users\DELL/.ssh/known_hosts:4: 192.168.1.73
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.11.144' (ED25519) to the list of known hosts.
design2ta@192.168.11.144's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-1048-raspi aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Mar  5 00:46:32 2026 from 192.168.1.74
design2ta@design2ta-desktop:~$ sudo apt install ros-jazzy-desktop
[sudo] password for design2ta:
```

II. After installation, you can verify by opening a Terminal and try this example.

Setup environment

Set up your environment by sourcing the following file.

```
# Replace ".bash" with your shell if you're not using bash
# Possible values are: setup.bash, setup.sh, setup.zsh
source /opt/ros/jazzy/setup.bash
```

In one terminal, source the setup file and then run a C++ `talker`:

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker
```

In another terminal source the setup file and then run a Python `listener`:

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

III. After installation, open ~/.bashrc by running this command:

nano ~/.bashrc

```
design2ta@design2ta-desktop: ~
design2ta@design2ta-desktop:~$ nano ~/.bashrc
```

Write the following at the end of the file, so you don't need to source setup.bash file of ROS jazzy (You can go down through the file using down arrow key):

source /opt/ros/jazzy/setup.bash

```
design2ta@design2ta-desktop: ~
  GNU nano 7.2                                        /home/design2ta/.b
alias l='ls -CF'

# Add an "alert" alias for long running commands.  Use like so:
#   sleep 10; alert
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo terminal
# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi
source /opt/ros/jazzy/setup.bash
# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
  if [ -f /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
  elif [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
  fi
fi
```

Save the file by pressing ***CTRL+S*** and then ***CTRL+X*** to close the file.

➢ *ROS2 Publisher and Subscriber Implementation*

Now, let's try to create a publisher and subscriber example on Raspberry Pi as first step

I. ROS2 Package

To create a package:

I. Change to src in your colcon workspace

cd ~/<ws_name>/src

II. Create the package using cmake build type (or python build type)

ros2 pkg create --build-type ament_cmake <pkg_name>

III. Build the package

a. Change to colcon workspace directory

cd ~/<ws_name>

b. Build the package

colcon build --symlink-install

IV. Configure the workspace environment in your command window

source install/setup.bash.

V. Create a scripts folder in the package

cd src/<pkg_name>

mkdir scripts

II. ROS2 Publisher and Subscriber

I. ROS2 Publisher

I. Inside scripts, create a python file for the publisher node

cd scripts

touch talker_node.py

II. Open talker_node.py file by running the following command and then paste the following code then save and close the file:

nano talker_node.py

```
#!/usr/bin/env python3
import rclpy
from std_msgs.msg import String

#define the global variables
node = None
publisher = None
i = 0


def timer_callback():
       global node, publisher, i
       msg = String()
       msg.data = "Hello World %d"%i
       publisher.publish(msg)
       node.get_logger().info('Publishing: "%s"'%msg.data)
       i+=1


def main(args=None):
       global node, publisher
       rclpy.init(args=args)
       node = rclpy.create_node("Publisher_Node")
       publisher = node.create_publisher(String, "Greeting_topic", 10)
       timer = node.create_timer(0.5, timer_callback)
       rclpy.spin(node)
       node.destroy_timer(timer)
       node.destroy_node()
       rclpy.shutdown()

if __name__ == "__main__":
       main()
```

```python
import rclpy
from std_msgs.msg import String

#define the global variables
node = None
publisher = None
i = 0


def timer_callback():
        global node, publisher, i
        msg = String()
        msg.data = "Hello World %d"%i
        publisher.publish(msg)
        node.get_logger().info('Publishing: "%s"'%msg.data)
        i+=1


def main(args=None):
        global node, publisher
        rclpy.init(args=args)
        node = rclpy.create_node("Publisher_Node")
        publisher = node.create_publisher(String, "Greeting_topic", 10)
        timer = node.create_timer(0.5, timer_callback)
        rclpy.spin(node)
        node.destroy_timer(timer)
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
        main()
```

II. ROS2 Subscriber

I. Inside scripts, create a python file for the subscriber node

cd scripts

touch listener_node.py

II. Open listener_node.py file by running the following command and then paste the following code then save and close the file:

nano listener_node.py

```
#!/usr/bin/env python3
import rclpy
from std_msgs.msg import String

node = None

def chatter_callback(msg):
        global node
        node.get_logger().info('Hearing: "%s"'%msg.data)


def main(args=None):
        global node
        rclpy.init(args=args)
        node = rclpy.create_node("Subscriber_Node")
        node.create_subscription(String, "Greeting_topic",
chatter_callback, 10)
        while rclpy.ok():
                rclpy.spin_once(node)
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
        main()
```

```
import rclpy
from std_msgs.msg import String

node = None

def chatter_callback(msg):
        global node
        node.get_logger().info('Hearing: "%s"'%msg.data)


def main(args=None):
        global node
        rclpy.init(args=args)
        node = rclpy.create_node("Subscriber_Node")
        node.create_subscription(String, "Greeting_topic", chatter_callback, 10)
        while rclpy.ok():
                rclpy.spin_once(node)
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
        main()
```

III. ROS2 Configurations

I. Make the files executable

chmod +x talker_node.py

chmod +x listener_node.py

II. In the package, you will find package.xml file which is important to declare all the dependencies of the package. We need to edit to add rclpy and std_msgs dependencies.

Open package.xml

cd ~/<ws_name>/src/<pkg_name>

nano package.xml

Add at the start of the dependencies

```
<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

```
design2ta@design2ta-desktop: ~/design_ws/src/lab5a
  GNU nano 7.2                                                package.xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schem
<package format="3">
  <name>lab5a</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="design2ta@todo.todo">design2ta</maintainer>
  <license>TODO: License declaration</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>std_msgs</exec_depend>


  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

III. We need to declare our nodes in CMakeLists.txt to be executable. Also, we need to define the scripts folder in share section

Open CMakeLists.txt

nano CMakeLists.txt

Add the following:

```
install(
  DIRECTORY scripts
  DESTINATION share/${PROJECT_NAME}
)

install(
  PROGRAMS
    scripts/talker_node.py
    scripts/listener_node.py

  DESTINATION lib/${PROJECT_NAME}
)
```

```
GNU nano 7.2                                                    CMakeLists.txt
cmake_minimum_required(VERSION 3.8)
project(lab5a)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
# uncomment the following section in order to fill in
# further dependencies manually.
# find_package(<dependency> REQUIRED)
install(
  DIRECTORY scripts
  DESTINATION share/${PROJECT_NAME}
)

install(
  PROGRAMS
    scripts/talker_node.py
    scripts/listener_node.py
  DESTINATION lib/${PROJECT_NAME}
)
if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  # the following line skips the linter which checks for copyrights
  # comment the line when a copyright and license is added to all source files
  set(ament_cmake_copyright_FOUND TRUE)
```

IV. Build the package and then source the environment of the workspace

cd ~/<ws_name>

colcon build --symlink-install

source install/setup.bash

IV. ROS2 Node Running

You can run the nodes in two cmd windows

ros2 run <pkg_name> talker_node.py

ros2 run <pkg_name> listener_node.py

```
design2ta@design2ta-desktop:~/design_ws$ ros2 run lab5a talker_node.py
[INFO] [1772664423.063241805] [Publisher_Node]: Publishing: "Hello World 0"
[INFO] [1772664423.542342717] [Publisher_Node]: Publishing: "Hello World 1"
[INFO] [1772664424.042176232] [Publisher_Node]: Publishing: "Hello World 2"
[INFO] [1772664424.542145906] [Publisher_Node]: Publishing: "Hello World 3"
[INFO] [1772664425.042096032] [Publisher_Node]: Publishing: "Hello World 4"
[INFO] [1772664425.542298556] [Publisher_Node]: Publishing: "Hello World 5"
[INFO] [1772664426.042230996] [Publisher_Node]: Publishing: "Hello World 6"
[INFO] [1772664426.542149447] [Publisher_Node]: Publishing: "Hello World 7"
[INFO] [1772664427.042105361] [Publisher_Node]: Publishing: "Hello World 8"
[INFO] [1772664427.542087864] [Publisher_Node]: Publishing: "Hello World 9"
[INFO] [1772664428.042097409] [Publisher_Node]: Publishing: "Hello World 10"
```

```
design2ta@design2ta-desktop:~/design_ws$ ros2 run lab5a listener_node.py
[INFO] [1772664440.063469893] [Subscriber_Node]: Hearing: "Hello World 34"
[INFO] [1772664440.542590845] [Subscriber_Node]: Hearing: "Hello World 35"
[INFO] [1772664441.042416030] [Subscriber_Node]: Hearing: "Hello World 36"
```

➢ *GPIO Configuration*

Now, we need to configure GPIOs on Raspberry Pi to connect IMU sensor to it

I. Install required packages for GPIOs

sudo apt update

sudo apt install gpiod libgpiod-dev python3-libgpiod

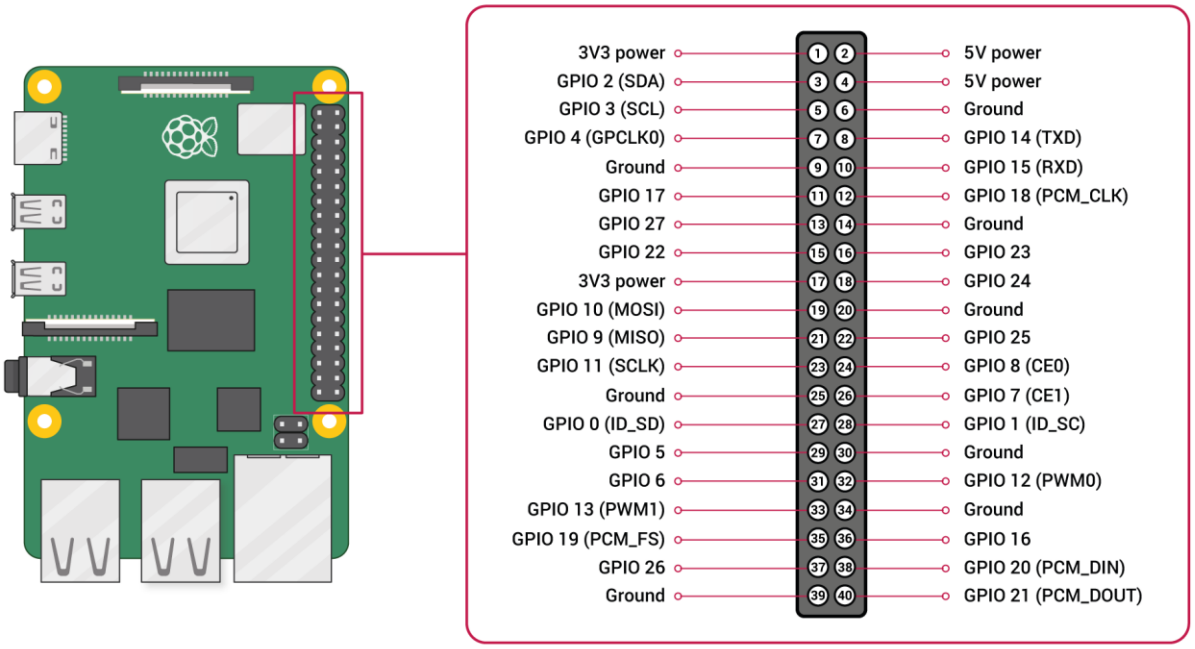
```
design2ta@design2ta-desktop:~$ sudo apt update
[sudo] password for design2ta:
Hit:1 http://ports.ubuntu.com/ubuntu-ports noble InRelease
Hit:2 http://packages.ros.org/ros2/ubuntu noble InRelease
Hit:3 http://ports.ubuntu.com/ubuntu-ports noble-updates InRelease
Hit:4 http://ports.ubuntu.com/ubuntu-ports noble-backports InRelease
Hit:5 http://ports.ubuntu.com/ubuntu-ports noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
design2ta@design2ta-desktop:~$ sudo apt install gpiod libgpiod-dev python3-libgpiod
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libgpiod2t64
Suggested packages:
  libgpiod-doc
The following NEW packages will be installed:
  gpiod libgpiod-dev libgpiod2t64 python3-libgpiod
0 upgraded, 4 newly installed, 0 to remove and 2 not upgraded.
Need to get 151 kB of archives.
After this operation, 1,133 kB of additional disk space will be used.
Do you want to continue? [Y/n]
```

II. Test the GPIOs packages installation

sudo gpioinfo

```
design2ta@design2ta-desktop:~$ sudo gpioinfo
gpiochip0 - 32 lines:
        line   0:          "-"       unused   input  active-high
        line   1: "2712_BOOT_CS_N" "spi10 CS0" output active-low [used]
        line   2: "2712_BOOT_MISO" unused input active-high
        line   3: "2712_BOOT_MOSI" unused input active-high
        line   4: "2712_BOOT_SCLK" unused input active-high
        line   5:          "-"       unused   input  active-high
        line   6:          "-"       unused   input  active-high
        line   7:          "-"       unused   input  active-high
        line   8:          "-"       unused   input  active-high
        line   9:          "-"       unused   input  active-high
        line  10:          "-"       unused   input  active-high
        line  11:          "-"       unused   input  active-high
        line  12:          "-"       unused   input  active-high
        line  13:          "-"       unused   input  active-high
        line  14:    "PCIE_SDA"      unused   input  active-high
        line  15:    "PCIE_SCL"      unused   input  active-high
        line  16:          "-"       unused   input  active-high
        line  17:          "-"       unused   input  active-high
        line  18:          "-"       unused   input  active-high
        line  19:          "-"       unused   input  active-high
        line  20:    "PWR_GPIO" "pwr_button"   input   active-low [used]
        line  21: "2712_G21_FS" unused input active-high
        line  22:          "-"       unused   input  active-high
        line  23:          "-"       unused   input  active-high
        line  24:      "BT_RTS"      unused   input  active-high
        line  25:      "BT_CTS"      unused   input  active-high
        line  26:      "BT_TXD"      unused   input  active-high
```

➢ *GPIO Pin Diagram*



➢ *GPIO Coding*

I. Inside the scripts directory, create a python file

cd ~/<ws_name>/src/<pkg_name>/scripts

touch gpio_led_test.py

nano gpio_led_test.py

II. Copy the following code in the file

```
# Blink the LED
import gpiod
import time

chip = gpiod.Chip('gpiochip4')  # Pi5 main GPIO chip

line = chip.get_line(17)  # GPIO17

line.request(consumer="led", type=gpiod.LINE_REQ_DIR_OUT)

while True:
    line.set_value(1)
    time.sleep(1)

    line.set_value(0)
    time.sleep(1)
```

```
#!/usr/bin/env python3
# Blink the LED
import gpiod
import time

chip = gpiod.Chip('gpiochip4')  # Pi5 main GPIO chip

line = chip.get_line(17)  # GPIO17

line.request(consumer="led", type=gpiod.LINE_REQ_DIR_OUT)

while True:
    line.set_value(1)
    time.sleep(1)

    line.set_value(0)
    time.sleep(1)
```

III. Now, connect the LED to GPIO17 (pin 11) and the ground of PI and run the code. The LED will blink now. (**N.B.** When running the command window for Raspberry PI ssh connection, run it as administrator)

python3 gpio_led_test.py

➢ *IMU Setup*

To work with IMU MPU6050 sensor, we will set up some plugins and work with I2C connection, and check for the connection of PI with IMU.

I. Install the required plugins for IMU and I2C connection

sudo apt update

sudo apt install i2c-tools

sudo apt install python3-smbus2

II. Now, connect IMU sensor. Connect as follows:

IMU PIN	RPI PIN
VCC	Pin 1 (3.3 V)
GND	Pin 6 (GND)
SDA	Pin 3 (GPIO2 “SDA”)
SCL	Pin 5 (GPIO3 “SCL”)

III. Now, load I2C driver and check for the address of IMU sensor

sudo modprobe i2c-dev

*ls /dev/i2c**

```
design2ta@design2ta-desktop:~$ sudo modprobe i2c-dev
[sudo] password for design2ta:
design2ta@design2ta-desktop:~$ ls /dev/i2c*
/dev/i2c-1  /dev/i2c-13  /dev/i2c-14
design2ta@design2ta-desktop:~$
```

Check for the address of the IMU sensor connected

sudo i2cdetect -y 1

The address of the connected IMU is ***0x68***.

```
design2ta@design2ta-desktop:~$ sudo i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

IV. Create a python script to get the readings from the IMU.

cd ~/<ws_name>/src/<pkg_name>/scripts

touch imu_readings.py

nano imu_readings.py

IV. Copy the following code in the file

```
import smbus2
import time

# MPU6050 I2C address
DEVICE_ADDR = 0x68

# Important Registers you got the addresses from datasheet
PWR_MGMT_1   = 0x6B
ACCEL_XOUT_H = 0x3B
GYRO_XOUT_H  = 0x43

# Initialize SMBus
bus = smbus2.SMBus(1) # 1 indicates /dev/i2c-1

def init_mpu():
    # Wake up the MPU6050 (it starts in sleep mode)
    bus.write_byte_data(DEVICE_ADDR, PWR_MGMT_1, 0)
    print("MPU6050 Initialized and Woken Up")

def read_raw_data(addr):
    # MPU6050 data is 16-bit (2 bytes), high byte first
    high = bus.read_byte_data(DEVICE_ADDR, addr)
    low = bus.read_byte_data(DEVICE_ADDR, addr + 1)

    # Combine high and low for a 16-bit value
    value = (high << 8) | low

    # Convert to signed value (handle negative numbers)
    if value > 32768:
        value = value - 65536
    return value

try:
    init_mpu()
    while True:
        # Read Accelerometer (Raw)
        acc_x = read_raw_data(ACCEL_XOUT_H)
        acc_y = read_raw_data(ACCEL_XOUT_H + 2)
        acc_z = read_raw_data(ACCEL_XOUT_H + 4)

        # Read Gyroscope (Raw)
        gyro_x = read_raw_data(GYRO_XOUT_H)
        gyro_y = read_raw_data(GYRO_XOUT_H + 2)
        gyro_z = read_raw_data(GYRO_XOUT_H + 4)

        print(f"ACCEL [X: {acc_x} Y: {acc_y} Z: {acc_z}] | GYRO [X: {gyro_x}
Y: {gyro_y} Z: {gyro_z}]")

        time.sleep(0.5)

except KeyboardInterrupt:
    print("\nStopping...")
```

V. Run the code and check for the readings from IMU. (**N.B.** You need to filter these readings to take the right readings)

python3 imu_readings.py

```
design2ta@design2ta-desktop:~/design_ws/src/lab5a/scripts$ sudo python3 imu_readings.py
MPU6050 Initialized and Woken Up
ACCEL [X: 2728 Y: -12844 Z: -7992] | GYRO [X: -911 Y: -245 Z: 155]
ACCEL [X: 2660 Y: -12852 Z: -7884] | GYRO [X: -897 Y: -74 Z: 15]
ACCEL [X: 2548 Y: -12892 Z: -7824] | GYRO [X: -906 Y: -81 Z: 28]
ACCEL [X: 2672 Y: -12952 Z: -7976] | GYRO [X: -903 Y: -81 Z: 23]
ACCEL [X: 2600 Y: -13028 Z: -7856] | GYRO [X: -932 Y: -81 Z: 110]
```